

- Description

This module manages a smart warehouse robot load monitoring and scheduling system, ensuring that:

1. **Each robot's load capacity is continuously monitored to prevent overloading.**
2. **The system optimizes task distribution among robots to balance workload and improve efficiency.**
3. **If a robot is overloaded, it stops accepting new tasks until its load is reduced.**
4. **A log of all robot loads, task assignments, and overload alerts is maintained for operational efficiency.**

This system helps improve warehouse automation, prevent equipment overload, and optimize task scheduling.

- Specification

module SmartWarehouseRobotMonitoring

var currentLoad, maxLoad, taskWeight: real

var log: sequence of string

process assignTask(taskWeight: real)

functional scenario form:

currentLoad + taskWeight <= maxLoad && currentLoad = currentLoad + taskWeight, log = log $\oplus$ ("Task Assigned: " + taskWeight + " kg | New Load: " + currentLoad + " kg"), Task assigned successfully.

||

currentLoad + taskWeight > maxLoad && log = log $\oplus$ ("Overload Warning: Task " + taskWeight + " kg rejected."), Overload warning triggered, task rejected.

end process

process completeTask(taskWeight: real)

functional scenario form:

taskWeight > 0 && currentLoad = max(0, currentLoad - taskWeight), log = log $\oplus$ ("Task Completed: " + taskWeight + " kg | Remaining Load: " + currentLoad + " kg"), Task completed, remaining load updated.

end process

end module

- Java Code

```
package code;
import java.util.*;
class WarehouseRobot {
    private String id;
    private double maxLoad;
    private double currentLoad;
    private List<String> log;
    public WarehouseRobot(String id, double maxLoad) {
```

```

        this.id = id;
        this.maxLoad = maxLoad;
        this.currentLoad = 0.0;
        this.log = new ArrayList<>();
    }
    public void assignTask(double taskWeight) {
        if (currentLoad + taskWeight > maxLoad) {
            System.out.printf("Robot %s Overloaded! Task Rejected.%n",
id);
                log.add(String.format("Overload Warning: Task %.1f kg rejected.", taskWeight));
                return;
            }
            currentLoad += taskWeight;
            System.out.printf("Robot %s Assigned Task: %.1f kg | Current Load: %.1f kg%n", id, taskWeight, currentLoad);
            log.add(String.format("Task Assigned: %.1f kg | New Load: %.1f kg", taskWeight, currentLoad));
        }
        public void completeTask(double taskWeight) {
            currentLoad = Math.max(0, currentLoad - taskWeight);
            System.out.printf("Robot %s Completed Task: %.1f kg | Remaining Load: %.1f kg%n", id, taskWeight, currentLoad);
            log.add(String.format("Task Completed: %.1f kg | Remaining Load : %.1f kg", taskWeight, currentLoad));
        }
        public void printLog() {
            System.out.printf("\nRobot %s Load Log:%n", id);
            for (String logEntry : log) {
                System.out.println(logEntry);
            }
        }
    }
}

public class c40_WarehouseRobotSystem {
    public static void main(String[] args) {
        // testcase-VT:
        // WarehouseRobot robot1 = new WarehouseRobot("R1", 50.0);
        // WarehouseRobot robot2 = new WarehouseRobot("R2", 60.0);
        // robot1.assignTask(20.0);
        // robot1.assignTask(35.0);
        // robot1.completeTask(10.0);
        // robot1.assignTask(30.0);
        // robot1.printLog();
        //
    }
}

```

```

//      robot2.assignTask(50.0);
//      robot2.completeTask(20.0);
//      robot2.assignTask(20.0);
//      robot2.assignTask(15.0);
//      robot2.printLog();

// testcase-FT:
//      WarehouseRobot robot1 = new WarehouseRobot("R1", 50.0);
//      robot1.assignTask(-4.9E-324);
//      robot1.assignTask(7.022238808055905E305);
//      robot1.assignTask(-1.7976931348623157E308);
//      robot1.assignTask(0.0);
//      robot1.assignTask(-9.376203409281882E307);
//      robot1.assignTask(-7.022238808055905E305);
//      robot1.assignTask(-1.0589518656958033E301);
//      robot1.assignTask(6.364956940265681E293);
//      robot1.printLog();

// testcase-Z3:
//      WarehouseRobot robot1 = new WarehouseRobot("R1", 50.0);
//////      WarehouseRobot robot2 = new WarehouseRobot("R2", 60.0);
//      robot1.assignTask(20.0);
//      robot1.assignTask(35.0);
//      robot1.completeTask(10.0);
//      robot1.assignTask(30.0);
//////      robot1.printLog();
//
//      robot1.assignTask(50.0);
//      robot1.completeTask(20.0);
//      robot1.assignTask(20.0);
//      robot1.assignTask(15.0);
//////      robot1.printLog();

// testcase-UVT:
WarehouseRobot robot1 = new WarehouseRobot("R1", 50.0);
WarehouseRobot robot2 = new WarehouseRobot("R2", 60.0);
robot1.assignTask(20.0);
robot1.assignTask(35.0);
robot1.completeTask(10.0);
robot1.assignTask(30.0);
robot1.printLog();
robot2.assignTask(50.0);
robot2.completeTask(20.0);
robot2.assignTask(20.0);

```

```

        robot2.assignTask(15.0);
        robot2.printLog();
    }
}

```

- Coverage report

1. Test case number: 8
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c40_WarehouseRobotSystem.java	100.0 %	4	0	4
> c40_WarehouseRobotSystem		0	0	0
> WarehouseRobot	100.0 %	4	0	4

3. Vibration testing instruction coverage: 98.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.0 %	216	20,465	20,681
▼ src	1.0 %	216	20,465	20,681
▼ code	1.0 %	216	20,465	20,681
▼ c40_WarehouseRobotSystem.java	98.6 %	216	3	219
> c40_WarehouseRobotSystem	93.2 %	41	3	44
> WarehouseRobot	100.0 %	175	0	175

4. Fuzz testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c40_WarehouseRobotSystem.java	100.0 %	4	0	4
> c40_WarehouseRobotSystem		0	0	0
> WarehouseRobot	100.0 %	4	0	4

5. Fuzz testing instruction coverage: 74.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.8 %	158	20,515	20,673
▼ src	0.8 %	158	20,515	20,673
▼ code	0.8 %	158	20,515	20,673
▼ c40_WarehouseRobotSystem.java	74.9 %	158	53	211
> WarehouseRobot	71.4 %	125	50	175
> c40_WarehouseRobotSystem	91.7 %	33	3	36

6. Z3 branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.2 %	2	838	840
▼ src	0.2 %	2	838	840
▼ code	0.2 %	2	838	840
▼ c40_WarehouseRobotSystem.java	50.0 %	2	2	4
> WarehouseRobot	50.0 %	2	2	4
> c40_WarehouseRobotSystem		0	0	0

7. Z3 instruction coverage: 85.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.8 %	179	22,193	22,372
▼ src	0.8 %	179	22,193	22,372
▼ code	0.8 %	179	22,193	22,372
▼ c40_WarehouseRobotSystem.java	85.6 %	179	30	209
> WarehouseRobot	84.6 %	148	27	175
> c40_WarehouseRobotSystem	91.2 %	31	3	34

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c40_WarehouseRobotSystem.java	100.0 %	4	0	4
> c40_WarehouseRobotSystem		0	0	0
> WarehouseRobot	100.0 %	4	0	4

9. Unimproved V-method instruction coverage: 98.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.0 %	216	20,465	20,681
▼ src	1.0 %	216	20,465	20,681
▼ code	1.0 %	216	20,465	20,681
▼ c40_WarehouseRobotSystem.java	98.6 %	216	3	219
> c40_WarehouseRobotSystem	93.2 %	41	3	44
> WarehouseRobot	100.0 %	175	0	175